Coinsult

Request your audit at **coinsult.net**

Advanced Manual

Smart Contract Audit

October 15, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Six Seven AI

0xdc59d7a20078622A2AD3836C5ddB60936360807e

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	4	4	0	0
Low-Risk	3	3	0	0
Medium-Risk	1	1	0	0
Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way
Low-Risk	Won't cause any problems, but can be adjusted for improvement
Medium-Risk	Will likely cause problems and it is recommended to adjust
Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Six Seven AI	
Website	sixsevenai.netlify.app
Blockchain	BNB Smart Chain (chain ID 56)
Source Code	contracts/SixSevenBurnVault.sol
Contract Address	0xdc59d7a20078622A2AD3836C5ddB60936360807e

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview 2
- Manual Code Review 2

Audit Summary 2

Audit Scope 3
- Audit Method 3
- Automated Vulnerability Check 3
- Manual Code Review 3
- Used tools 3

Table of Contents 4

● There Is Genuinely No Route From The Reserve To Anywhere But The Dead Address

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
VLT-I1	The header claim was checked against every function. The only transfer of the configured token is to the fixed dead address, the rescue function refuses that token outright, and the token address is set once at deployment and cannot be changed. The token itself is a single contract rather than one behind a proxy, so there is no second address that would slip past the rescue check. The claim holds.

Code Snippet

```
IERC20 public immutable token;
address public constant DEAD = 0x...dEaD;

function rescueForeign(address foreign, address to, uint256 amount) ... {
    require(foreign != address(token), "cannot touch reserve");
```

Recommendation

No change needed. Keep the token address immutable if the contract is ever revised.

The Burn Counter Records What Was Sent, Not What Arrived

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
VLT-I2	The counter adds the amount passed to the transfer rather than measuring how much the dead address actually received. With the current token the two are the same because this transfer is not taxed, so the figure is accurate today. It would overstate the burn if the token were ever changed or if the vault were ever treated as an exchange pair.

Code Snippet

```
require(token.transfer(DEAD, amount), "transfer failed");
totalBurned += amount;    // not measured against the recipient
```

Recommendation

Measure the dead address balance before and after the transfer and add the difference, which keeps the published figure true under any token behaviour.

Anyone Can Add To The Reserve And Nobody Can Take It Out

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
VLT-I3	Any $67AI sent to this address becomes part of the reserve and can only ever be burned, whoever sent it and whether or not they meant to. That follows directly from the design and is worth stating plainly, because a holder who transfers tokens here by mistake has no way to recover them.

Code Snippet

```
function reserveBalance() external view returns (uint256) {
    return token.balanceOf(address(this));
}
// burnAll() burns whatever the address holds
```

Recommendation

State on the website that this address accepts no refunds, so holders do not treat it as an ordinary project wallet.

The Reserve Matches The Published Figure

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
VLT-I4	At the reviewed block the vault holds exactly sixty million tokens against an initial supply of one hundred million, which is the sixty percent shown in the published tokenomics. The separate figure the website gives for tokens already burned comes from the token's own trading tax and not from this contract.

Code Snippet

```
vault balance   = 60000000 tokens
initial supply  = 100000000 tokens
current supply  =  99135711.94 tokens  (token tax burns, not this vault)
totalBurned     = 0
```

Recommendation

Keep the two figures separate in public material so the trading tax burn is not read as progress against the sixty percent reserve.

The Vault Is Not Marked Exempt On The Token

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
VLT-L1	Unlike the staking contract, this vault is not on the token's exempt list, so each burn runs through the token's full transfer path rather than skipping it. It works today for two separate reasons: the token only charges tax when one side is an exchange pair, and the dead address is on the token's limit exempt list so the wallet cap is not applied. Both of those are settings the token owner can change, and if the dead address ever lost its limit exemption a burn of the full reserve would exceed the four hundred thousand wallet cap and revert.

Code Snippet

```
// reviewed state on the token
isFeeExempt[vault]   = false
isLimitExempt[vault] = true
isFeeExempt[DEAD]    = false
isLimitExempt[DEAD]  = true     <- the only reason a large burn passes
```

Recommendation

Call setFeeExempt on the token for this vault address, which makes its transfers bypass the token's checks entirely and removes the dependency on how the dead address is configured.

Ownership Transfer Is One Step With No Confirmation

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
VLT-L2	Ownership moves to whatever address is passed in, with only a check that it is not the zero address. Because the owner is the only party who can ever burn, a mistyped or unreachable address would leave sixty percent of the supply locked in the vault with no route out for anyone, including the project. The current owner is an ordinary account carrying a delegation to a MetaMask smart account implementation, so one key stands behind the transfer.

Code Snippet

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), "owner=0");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

Recommendation

Use a two step transfer where the new owner has to accept, and hold the role in a multiple signature wallet.

● Transfers Are Made Without The Safe Wrapper Used Elsewhere

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
VLT-L3	The vault calls transfer directly and checks the returned flag, while the companion staking contract uses the SafeERC20 wrapper for the same job. It behaves correctly against the configured token, which returns a flag as the standard expects. It would revert against a token that returns nothing, which is a real pattern among older tokens, so the two contracts in this suite handle the same situation differently.

Code Snippet

```
require(token.transfer(DEAD, amount), "transfer failed");

// staking contract, same operation
stakingToken.safeTransfer(msg.sender, amount);
```

Recommendation

Use SafeERC20 here as well so both contracts behave the same way and the vault stays correct if the configured token is ever changed.

The Burn Is Discretionary With No Schedule Or Deadline

Medium-Risk *- Will likely cause problems and it is recommended to adjust* **[Status: Pending]**

Error Code	Description
VLT-M1	The contract does enforce the part that matters most, which is that the reserve can only ever leave towards the dead address and cannot be sold. What it does not do is commit to burning anything at any point. There is no schedule, no deadline, no minimum and no way for anyone other than the owner to act, so the sixty million tokens can sit in place indefinitely. At the reviewed block the counter still reads zero, meaning none of the reserve has been burned since deployment.

Code Snippet

```
function burn(uint256 amount) external onlyOwner { ... }
function burnAll() external onlyOwner { ... }

// reviewed state
reserveBalance = 60000000 tokens   (60% of the 100M initial supply)
totalBurned    = 0
```

Recommendation

Publish a burn schedule and enforce it in the contract, for example by allowing anyone to trigger the burn of a tranche once its date has passed, so the outcome does not rest on one party continuing to act.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.